

lab2

2026-01-29

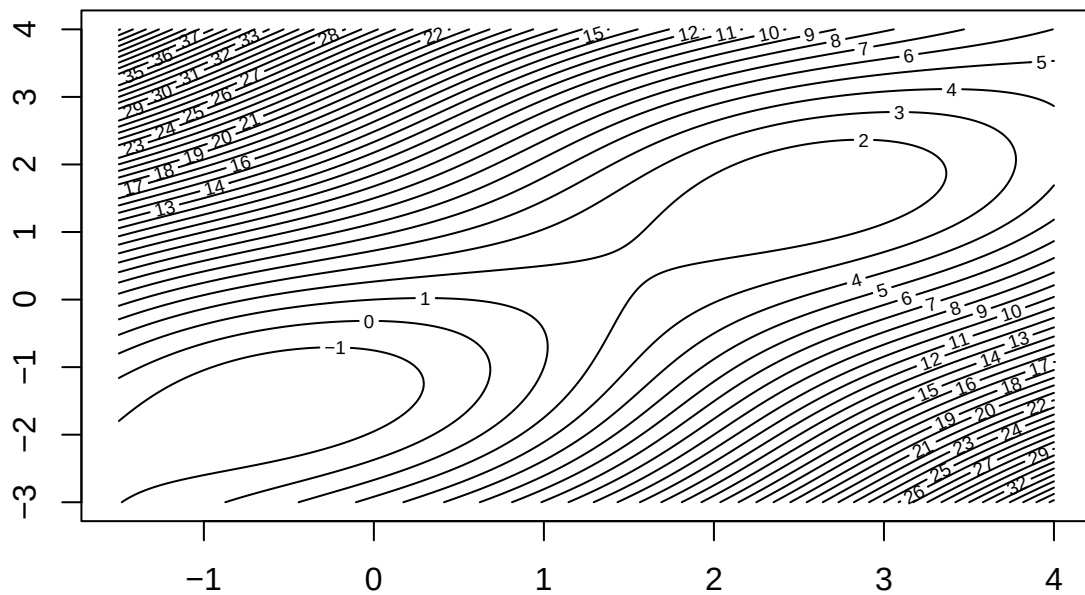
a

```
f <- function(x, y) {  
  sin(x+y) + (x-y)^2 - 1.5*x + 2.5*y + 1  
}
```

```
# y <- -pi/3 - 0.5  
# x <- y + 1  
# f(x, y)  
f(3, 2)
```

```
## [1] 1.541076
```

```
y <- seq(-3, 4, length.out = 500)  
x <- seq(-1.5, 4, length.out = 500)  
# outer creates all combinations of x, y  
# applying f to the matrix based on x and y  
z <- outer(x, y, f)  
  
contour(x = x, y = y, z = z, nlevels = 40)
```



b

```
#sin(x+y) + (x-y)^2 - 1.5*x + 2.5*y + 1

g <- function(v) {
  x <- v[1]
  y <- v[2]
  d_dx <- cos(x+y) + 2*x - 2*y - 1.5
  d_dy <- cos(x+y) + 2*y - 2*x + 2.5
  return( c(d_dx, d_dy) )
}

H <- function(v) {
  x <- v[1]
  y <- v[2]
  d_d2x <- -sin(x+y) + 2
  d_d2y <- -sin(x+y) + 2
  d_dxy <- -sin(x+y) - 2

  return(matrix(c(d_d2x, d_dxy, d_dxy, d_d2y), 2, 2))
}

g(c(0,0))
```

```
## [1] -0.5  3.5
```

```
H(c(0,0))
```

```
##      [,1] [,2]
## [1,]    2  -2
## [2,]  -2   2
```

c

Newton is finding points where the functions gradient is 0, saddle, local minima or local maxima. $x_{\text{new}} = x_{\text{old}} - H^{-1} * \text{grad}$

```
Newton <- function(start, eps = 10^(-6), max_it = 50, x_boundry = c(-1.5, 4), y_boundry = c(-3, 4)) {
  L2 <- function(v) {
    sqrt(sum(v^2))
  }

  x_t <- start
  itterations <- 0
  stop_criteria <- T

  while (
    stop_criteria & itterations < max_it
  ) {
    itterations <- itterations + 1
    x_prev <- x_t

    # For some values the hessian was singular, solving this by adding a small lambda * I to the hessian
    #browser()
    H_inv <- tryCatch({solve(H(x_prev))},
      error = function(x) {solve(H(x_prev) + eps * diag(length(x_t)))})

    x_t <- x_prev - H_inv %*% g(x_prev)
    stop_criteria <- L2(x_t - x_prev) / (eps + L2(x_prev)) > eps
  }

  if (x_t[1] >= x_boundry[1] & x_t[1] <= x_boundry[2] & x_t[2] >= y_boundry[1] & x_t[2] <= y_boundry[2]) {
    eigen_vals <- eigen(H(x_t))$values
    cat("Start:", start, ", Itterations:", itterations, ", solution:", x_t, ", function value:", f(x_t[1]
  } else {
    cat("Solution outside the boundry", "\n\n")
  }
}
```

```

    return(x_t)
}

```

d

```

grid <- expand.grid(x = seq(-1.5, 4, length.out = 3), y = seq(-3, 4, length.out = 3))

for (i in 1:nrow(grid)) {
  v <- c(grid[i, 1], grid[i, 2])
  Newton(v)
}

## Solution outside the boundry
##
## Start: 1.25 -3 , Itterations: 4 , solution: -0.5471976 -1.547198 , function value: -1.913223
## eigen_values: 4 1.732051
##
## Start: 4 -3 , Itterations: 5 , solution: 1.547198 0.5471976 , function value: 1.913223
## eigen_values: 4 -1.732051
##
## Start: -1.5 0.5 , Itterations: 5 , solution: -0.5471976 -1.547198 , function value: -1.913223
## eigen_values: 4 1.732051
##
## Start: 1.25 0.5 , Itterations: 4 , solution: 1.547198 0.5471976 , function value: 1.913223
## eigen_values: 4 -1.732051
##
## Start: 4 0.5 , Itterations: 4 , solution: 2.594395 1.594395 , function value: 1.22837
## eigen_values: 4 1.732051
##
## Start: -1.5 4 , Itterations: 4 , solution: 1.547198 0.5471976 , function value: 1.913223
## eigen_values: 4 -1.732051
##
## Start: 1.25 4 , Itterations: 5 , solution: 2.594395 1.594395 , function value: 1.22837
## eigen_values: 4 1.732051
##
## Solution outside the boundry

```

e

If $v^T H v < 0$ we have a maximum. Loose proof sketch: $v^T H v$ is a dotproduct, $H v$ is the gradient in that direction. If the gradient, (which points in towards the highest ascent direction), points in the opposite direction as our step, then we are going downhill. So if all our steps are downhill, negative definite hessian means local maximum. Positive definite means local minimum. If all eigen values are negative we have a local maxima, all are positive means local minimi. Both means saddle point. We see we only found saddle points or local minima in our optimization.

This optimization searches unbounded but filters out solutions that lie outside the bounds. Starting points are a grid of 9 points inside the constraints. All points where the gradient is 0 were found, see appendix for an analytical solution.

As the function is bounded we in general check the value at all local minima aswell as the boundary-points to obtain the minima given our constraints. Given the contour plot the minima does not lie on the boundary.

Q 2.

a

```
# creating the data
x <- c(0, 0, 0, 0.1, 0.1, 0.3, 0.3, 0.9, 0.9, 0.9)
y <- c(0, 0, 1, 0, 1, 1, 1, 0, 1, 1)

g <- function(v) {
  b0 <- v[1]
  b1 <- v[2]

  sum( y * log( 1 / (1 + exp(-b0 -b1*x)) ) + (1 - y) * log( 1 - ( 1 / (1 + exp(-b0 -b1*x)) ) ) )
}

dg1 <- function(v) {
  b0 <- v[1]
  b1 <- v[2]

  sum( y - 1 / (1 + exp(-b0 -b1*x)) )
}

dg2 <- function(v) {
  b0 <- v[1]
  b1 <- v[2]

  sum( (y - 1 / (1 + exp(-b0 -b1*x))) * x )
}

# The function g to be maximised, partial derivatives dg1, dg2, and gradient
gradient <- function(x)
{
  c(dg1(x), dg2(x))
}

GD_optimizer <- function(x0, x = c(0, 0, 0, 0.1, 0.1, 0.3, 0.3, 0.9, 0.9, 0.9), y = c(0, 0, 1, 0, 1, 1, 1, 0, 1, 1),
                           backtracking_shrinkage = 0.5) {
  # Produce a contour plot; define first a grid where function is evaluated
  x1grid <- seq(-2, 2.5, by=0.05)
  x2grid <- seq(-2, 3, by=0.05)
  dx1 <- length(x1grid)
  dx2 <- length(x2grid)
  dx <- dx1*dx2
  gx <- matrix(rep(NA, dx), nrow=dx1)
  for (i in 1:dx1)
  for (j in 1:dx2)
  {
    gx[i,j] <- g(c(x1grid[i], x2grid[j]))
  }
  mgx <- matrix(gx, nrow=dx1, ncol=dx2)
```

```

contour(x1grid, x2grid, mgx, nlevels=34) # Note: For other functions g, you might need to choose another

#Steepest ascent function:
steepestasc <- function(x0 = x0, eps=1e-6, alpha0=1)
{
  L2 <- function(v) {
    sqrt(sum(v^2))
  }

  fun_calls <- 0
  grad_calls <- 0
  xt <- x0
  conv <- 999
  points(xt[1], xt[2], col=2, pch=4, lwd=3)
  while(conv>eps)
  {
    alpha <- alpha0

    # xt1 is xtold
    xtold <- xt
    xt <- xtold + alpha*gradient(xtold)
    grad_calls <- grad_calls + 1

    gxt <- g(xt)
    g_old <- g(xtold)
    fun_calls <- fun_calls + 2

    # modified to make less function calls
    while (gxt < g_old)
    {

      g_old <- gxt

      alpha <- alpha * backtracking_shrinkage
      xt <- xtold + alpha*gradient(xtold)

      gxt <- g(xt)
      fun_calls <- fun_calls + 2

    }
    points(xt[1], xt[2], col=2, pch=4, lwd=1)

    #
    conv <- L2(xt - xtold)
  }
  points(xt[1], xt[2], col=4, pch=4, lwd=3)
  cat("Function calls:", fun_calls, "\n")
  cat("Gradient calls:", grad_calls, "\n")

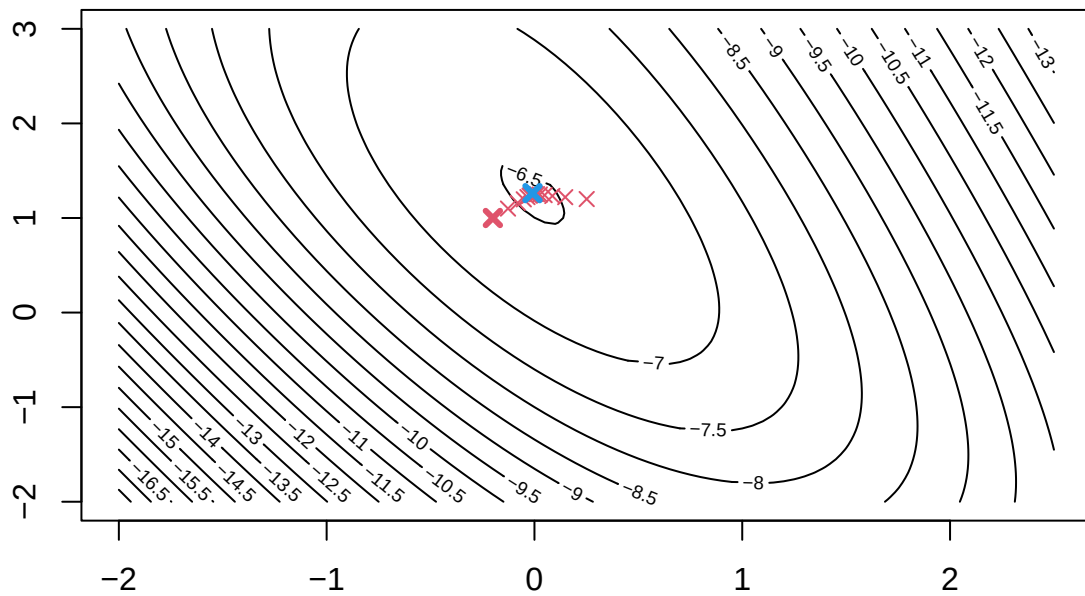
  return(xt)
}

```

```
steepestasc(c(-0.2, 1))
```

```
}
```

```
GD_optimizer(c(-0.2, 1), backtracking_shrinkage = 0.7)
```

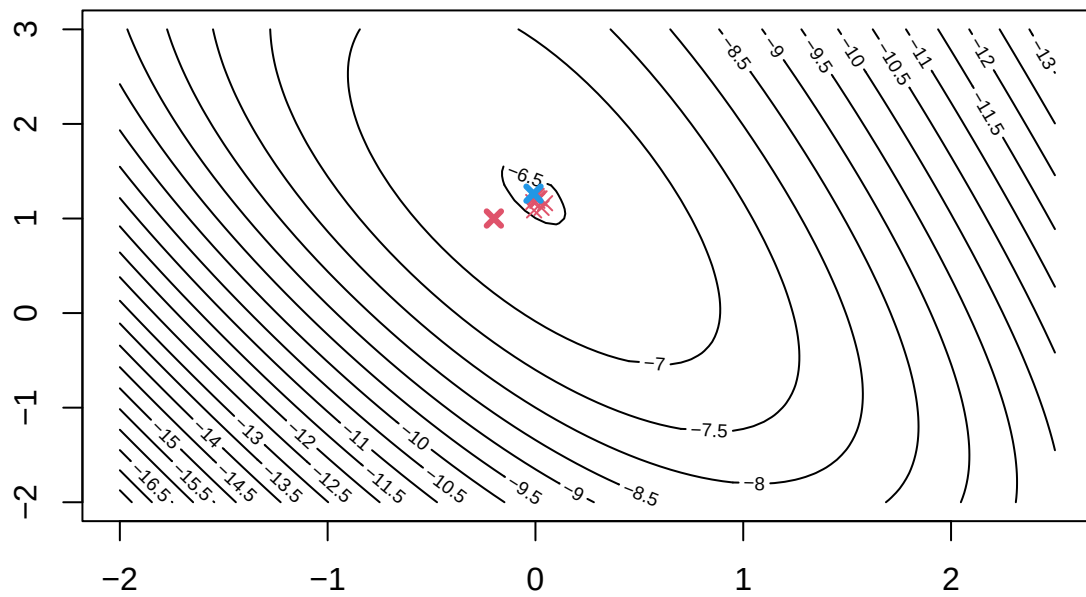


```
## Function calls: 224
```

```
## Gradient calls: 57
```

```
## [1] -0.009359101 1.262822253
```

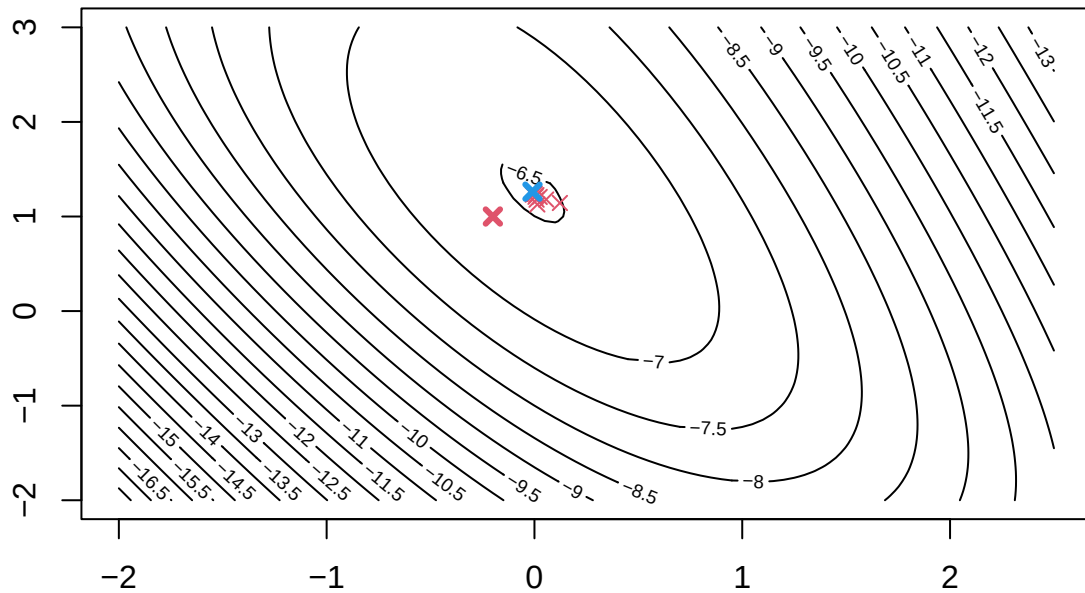
```
GD_optimizer(c(-0.2, 1), backtracking_shrinkage = 0.3)
```



```
## Function calls: 138
## Gradient calls: 50
```

```
## [1] -0.009358557  1.262820302
```

```
GD_optimizer(c(-0.2, 1), backtracking_shrinkage = 0.5)
```



```
## Function calls: 138
## Gradient calls: 49
```

```
## [1] -0.009359121  1.262821823
```

We set the change between x_t and x_{told} to be smaller than 10^{-6} so we can trust 5 decimals. With different backtracking methods we get way more function calls if the shrinkage factor for α is larger there needs to be more calls made. 0.3 and 0.5 perform similarly.

c

```
# optim minimizes so we try to maximize by multiplying by -1 via fnscale
optim(c(-0.2, 1), fn = g, gr = gradient, method = "BFGS", control = list(fnscale = -1))
```

```
## $par
## [1] -0.009356126  1.262812832
##
## $value
## [1] -6.484279
##
## $counts
## function gradient
##      12      8
```

```
##
## $convergence
## [1] 0
##
## $message
## NULL
```

```
optim(c(-0.2, 1), fn = g, method = "Nelder-Mead", control = list(fnscale = -1))
```

```
## $par
## [1] -0.009423433  1.262738266
##
## $value
## [1] -6.484279
##
## $counts
## function gradient
##      47      NA
##
## $convergence
## [1] 0
##
## $message
## NULL
```

They converge way faster and therefore use fewer function-calls. Nelder-Mead converging faster is interesting as it has no theoretical convergence speed.

Our result from b: -0.00935(9121) 1.26282(1823) precise to the 5th decimal. BFGS appears to have 4 decimals of precision and nelson meader appears to have 3. Digging into the documentation we find that the relative tolerance is $1e-8$, so the amount of decimals depends on the size of the root.

d

```
model <- glm(y ~ x, family = "binomial")
a <- summary(model)
a$coefficients[, 1]
```

```
## (Intercept)      x
## -0.009359853  1.262823430
```

It matches well with our optimized results for the first 5 decimals.

Appendix

1 increase -0.009354850 1.262808 -6.484279 40 58 40
 ## 2 armijo -0.009355336 1.262808 -6.484279 40 58 40
 where n_g is the number of function evaluations and n_{grad} is the number of gradient evaluations.
 We have implemented two different backtracking line search methods: increase-only backtracking ("inc")
 and Armijo backtracking ("armijo"). Both methods choose a step size $\alpha > 0$ for the update $b_{new} =$

$$x \in [-1.5, 4] \quad y \in [-3, 4]$$

$$\cos(x+y) + 2(x-y) - 1.5 = 0 \quad (1)$$

$$\cos(x+y) - 2(x-y) + 2.5 = 0 \quad (2)$$

$$x+y=u \Rightarrow \cos u + 2v - 1.5 = 0 \quad (1)$$

$$v=x-y \Rightarrow \cos u - 2v + 2.5 = 0 \quad (2)$$

$$(1)+(2) \Leftrightarrow \cos u = -\frac{1}{2} \Rightarrow$$

$$u = \pm \frac{2\pi}{3} + 2n\pi = u$$

$$(1)-(2) \Leftrightarrow 4v - 4 = 0 \Leftrightarrow v = 1 \Leftrightarrow x = y + 1$$

$$x+y = 2y+1 = u \Leftrightarrow y = \frac{u}{2} - \frac{1}{2} \Rightarrow$$

$$y = n\pi + \frac{\pi}{3} - \frac{1}{2} \rightarrow h = \{ -1, 0, 1 \}$$

$$y = n\pi - \frac{\pi}{3} - \frac{1}{2} \rightarrow h = \{ 0, 1 \}$$

$$x = y+1 \Rightarrow x > 4$$

$$y = \frac{\pi}{3} - \frac{1}{2}, x = \frac{\pi}{3} + \frac{1}{2}, f = 1.913$$

$$y = -\frac{2\pi}{3} - \frac{1}{2}, x = -\frac{2\pi}{3} + \frac{1}{2}, x < -1.5$$

Min at
 $x = -0.547$
 $y = -1.547$

$$y = -\frac{\pi}{3} - \frac{1}{2}, x = -\frac{\pi}{3} + \frac{1}{2}, f = -1.913$$

$$y = \frac{2\pi}{3} - \frac{1}{2}, x = \frac{2\pi}{3} + \frac{1}{2}, 1.228$$

Figure 1: Analytical solution Q1